

ANKARA UNIVERSITY
FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING



BLM4522 Ağ Tabanlı Paralel Tanıtım Sistemleri

Proje 2: Veri tabanı Yedekleme ve Felaketten Kurtarma Planı

Tuğrul Özgün

22290082

Git link: <https://github.com/Forcipus/Paralel-Sistemler>

Video link:

[https://drive.google.com/drive/folders/1nSic684UgTXflci_Lcs9nr0GDjoYQrYq?
usp=sharing](https://drive.google.com/drive/folders/1nSic684UgTXflci_Lcs9nr0GDjoYQrYq?usp=sharing)

İÇİNDEKİLER

1. GİRİŞ VE AMAÇ 2

1.1. Kapsam ve Metodolojisi

2. YEDEKLEME STRATEJİLERİNİN OLUŞTURULMASI 2

2.1. Tam Yedekleme (Full Backup) Uygulaması

2.2. Fark Yedeklemesi (Differential Backup) Uygulaması

2.3. Artık/İşlem Günlüğü Yedeklemesi (Transaction Log Backup)

3. ZAMANLAYICILARLA YEDEKLEME VE OTOMASYON (SQL SERVER AGENT) 4

3.1. SQL Server Agent Servisinin Yapılandırılması

3.2. Yedekleme Görevinin (Job) Oluşturulması

3.3. Zamanlama (Schedules) Stratejisinin Kurulması

4. FELAKETTEN KURTARMA SENARYOLARI 5

4.1. Senaryonun Hazırlanması (Felaket Öncesi)

4.2. Felaket Anı: Verinin Yanlışlıkla Silinmesi (Kaza)

4.3. Kurtarma Süreci: Point-in-Time Restore (Zamanda Geriye Yolculuk)

5. TEST YEDEKLEME SENARYOLARI: YEDEKLERİN DOĞRULUĞUNU TEST ETME 7

5.1. T-SQL ile Teknik Doğrulama (RESTORE VERIFYONLY)

5.2. SSMS Arayüzü ile Otomatik Doğrulama

5.3. Sayfa Doğruluğu (Checksum)

6. SONUÇ 8

1. GİRİŞ VE AMAÇ

Günümüz bilgi sistemlerinde veritabanları, kurumların e değerli dijital varlıklarını barındırmaktadır. Veri kaybı veya sistem kesintileri; donanım arızaları, yazılım hataları, siber saldırılar veya insan kaynaklı operasyonel hatalar gibi birçok nedenden kaynaklanabilmektedir. Bu nedenle, bir Veritabanı Yöneticisi (DBA) için en kritik sorumluluk, verinin erişilebilirliğini ve bütünlüğünü koruyacak kapsamlı bir Felaketten Kurtarma Planı (Disaster Recovery Plan) oluşturmaktır.

Bu projenin temel amacı, **Microsoft SQL Server** ortamında profesyonel yedekleme (backup) ve geri yükleme (restore) tekniklerini uygulamalı olarak simüle etmektir. Çalışma kapsamında, kurumsal düzeydeki veri setlerini içeren **WideWorldImporters** veritabanı kullanılarak; sistemin iş sürekliliğini sağlamak, veri kaybı toleransını (RPO) minimuma indirmek ve olası bir felaket anında sistemi saniyeler içinde eski kararlı haline döndürmek hedeflenmiştir.

1.1. Kapsam ve Metodoloji

Proje süresince aşağıdaki temel veritabanı yönetim süreçleri gerçekleştirilmiştir:

- Yedekleme Stratejileri:** Veri büyüklüğü ve değişim sıklığı göz önüne alınarak Tam (Full), Fark (Differential) ve İşlem Günlüğü (Transaction Log) yedekleme türleri yapılandırılmıştır.
- Otomasyon ve İş Sürekliliği:** SQL Server Agent servisi kullanılarak, insan müdahalesine gerek duymayan periyodik yedekleme işleri (jobs) ve zamanlayıcılar (schedules) kurulmuştur.
- Zaman Odaklı Kurtarma (Point-in-Time Restore):** Veritabanının sadece son yedek anına değil, Transaction Log yedekleri sayesinde felaketin gerçekleştiği saniyenin hemen öncesine döndürülmesi simüle edilmiştir.
- Güvenilirlik Testleri:** Alınan yedeklerin fiziksel bütünlüğü RESTORE VERIFYONLY ve CHECKSUM yöntemleri ile doğrulanmıştır.

Bu raporda, söz konusu tekniklerin uygulama adımları, karşılaşılan teknik engeller (kısıtlamalar, erişim hataları vb.) ve bu engellere yönelik geliştirilen mühendislik çözümleri detaylı bir şekilde dökümanite edilmiştir.

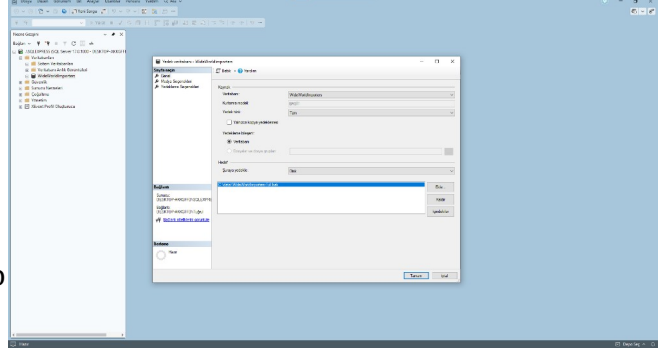
2. YEDEKLEME STRATEJİLERİNİN OLUŞTURULMASI

Veri bütünlüğünü korumak ve olası bir felaket anında veri kaybını sıfıra indirmek amacıyla Full (Tam), Differential (Fark) ve Transaction Log (Artık/Günlük) yedekleme türlerini kapsayan bir strateji izlenmiştir. Bu üçlü yapı, hem depolama alanından tasarruf etmeyi hem de veritabanını belirli bir zaman dilimine (Point-in-Time) döndürebilmeyi sağlar.

2.1. Tam Yedekleme (Full Backup) Uygulaması

Veritabanındaki tüm verilerin ve yapıların eksiksiz bir kopyasını almak için şu adımlar izlenmiştir:

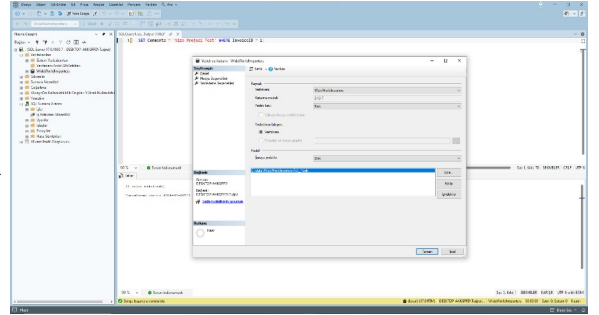
- **Erişim:** SSMS (SQL Server Management Studio) üzerinden **WideWorldImporters** veritabanına sağ tıklanarak **Tasks > Back Up** menüsüne girilmiştir.
- **Yapılandırma:** Yedekleme türü (Backup type) olarak **Full** seçilmiştir.
- **Medya Yönetimi:** "Media Options" sekmesinden "**Overwrite all existing backup sets**" seçeneği işaretlenerek dosya çakışmaları önlenmiştir.
- **Sıkıştırma:** "Options" sekmesinde, sistem kaynaklarını ve sürüm kısıtlamalarını göz önüne alarak "**Uncompressed**" (Sıkıştırılmamış) seçeneği ile işlem tamamlanmıştır.



2.2. Fark Yedeklemesi (Differential Backup) Uygulaması

Sadece son tam yedekten bu yana değişen verileri yedeklemek için şu adımlar izlenmiştir:

- **Süreç:** Tekrar **Back Up** menüsüne girişi yapılarak veritabanı bağlamı seçilmiştir.
- **Tür Seçimi:** Yedekleme türü olarak **Differential** seçilerek, dosya boyutunun küçük tutulması ve yedekleme süresinin kısaltılması sağlanmıştır.
- **Amaç:** Bu yedek türü, felaket anında geri yükleme hızını artırmak amacıyla stratejiye dahil edilmiştir.



2.3. Artık/İşlem Günlüğü Yedeklemesi (Transaction Log Backup)

Veritabanındaki her bir işlemi (transaction) kaydeden bu yedekleme türü, "Point-in-Time Restore" işleminin temelini oluşturur:

- **Ön Koşul:** İşlemin yapılabilmesi için veritabanı kurtarma modeli (Recovery Model) "**Full**" olarak ayarlanmıştır.
- **Uygulama:** Yedekleme tipi olarak "**Transaction Log**" seçilerek, veritabanında yapılan son değişikliklerin günlüğü tutulmuştur.

- **Amaç:** Bu yedekleme tipi, veritabanını felaketten (örneğin hatalı bir tablo silme işlemi) hemen önceki dakikaya döndürmek için "Timeline" özelliği ile birlikte kullanılmıştır

3. ZAMANLAYICILARLA YEDEKLEME VE OTOMASYON (SQL SERVER AGENT)

Veritabanı yönetiminde yedekleme işlemlerinin manuel olarak yapılması, insan hatasına açık ve sürdürülebilir olmayan bir yöntemdir. Bu nedenle, iş sürekliliğini sağlamak adına yedekleme süreçleri otomatize edilmiştir.

3.1. SQL Server Agent Servisinin Yapılandırılması

Otomasyonun temel motoru olan SQL Server Agent, arka planda zamanlanmış görevleri (Jobs) yürüten bir Windows servisi.

- **Aktivasyon:** SSMS Object Explorer panelinin en altında yer alan SQL Server Agent simgesine sağ tıklayarak "Start" komutu verilmiş ve servisin faal duruma geçmesi sağlanmıştır. Bu adım, otomasyonun çalışması için ön koşuldur.

3.2. Yedekleme Görevinin (Job) Oluşturulması

Otomatik yedekleme işlemini gerçekleştirecek olan iş birimi şu adımlarla tanımlanmıştır:

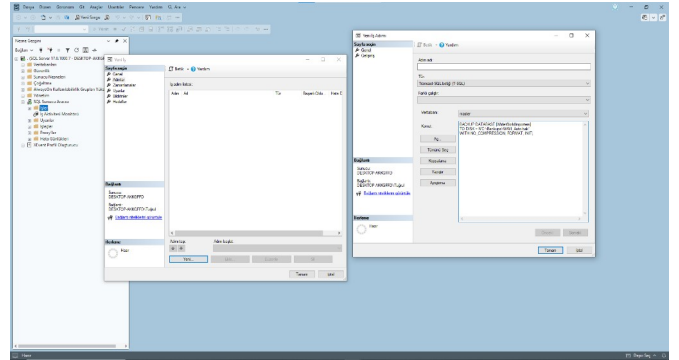
1. **Job Tanımı:** Jobs klasörüne sağ tıklayarak **New Job** seçeneği ile yeni bir görev başlatılmıştır. "General" sekmesinde işin adı profesyonel isimlendirme standartlarına uygun olarak **Daily_Full_Backup** olarak belirlenmiştir.

2. **İş Adımı (Step) Konfigürasyonu:** "Steps" sekmesinde görevin teknik içeriği

tanımlanmıştır. Sistemin her tetiklendiğinde yürüteceği T-SQL komutu şu şekildedir:

```
BACKUP DATABASE [WideWorldImporters]
TO DISK = N'C:\Backups\WWI_Auto.bak'
WITH NO_COMPRESSION, FORMAT, INIT;
```

- **FORMAT ve INIT:** Bu parametreler, her yeni yedeklemede dosyanın üzerine yazılmasını ve medya setinin yeniden başlatılmasını sağlayarak dosya boyutunun kontrolsüz büyümesini engeller.
- **NO_COMPRESSION:** Sürüm uyumluluğu ve işlemci yükünü optimize etmek amacıyla tercih edilmiştir.



3.3. Zamanlama (Schedules) Stratejisinin Kurulması

Oluşturulan yedekleme görevinin hangi periyotlarla çalışacağı "Schedules" sekmesinde kararlaştırılmıştır:

- **Zamanlayıcı Ayarı:** "New" butonuna basılarak oluşturulan zamanlayıcıda, yedekleme işleminin **her gün (Daily)** ve sistem yükünün en az olduğu **gece saat 02:00**'de çalışması planlanmıştır.

Bu zamanlama stratejisi, olası bir felaket durumunda kaybedilecek maksimum veri miktarını (RPO) 24 saat ile sınırlandırarak iş sürekliliği hedeflerine hizmet eder.

4. FELAKETTEN KURTARMA SENARYOLARI

Veritabanı yönetiminde en kritik aşama, beklenmedik veri kayıpları veya hatalı operasyonlar sonrası sistemin en az veri kaybıyla ayağa kaldırılmasıdır. Bu bölümde, bir tablonun tamamen silindiği bir "felaket" senaryosu ve bu durumdan Point-in-Time Restore (Zamanda Belirli Bir Ana Dönme) tekniği ile kurtulma süreci ele alınmıştır.

4.1. Senaryonun Hazırlanması (Felaket Öncesi)

Veriyi saniyeler bazında hassasiyetle kurtarabilmek için sistemin yapısal olarak buna hazır olması gerekir:

- **Kurtarma Modeli (Recovery Model):** Veritabanının "Point-in-Time Restore" özelliğini desteklemesi için **Recovery Model** ayarının **Full** (Tam) olduğundan emin olunmuştur (Sağ tık > Properties > Options). Bu ayar, yapılan her işlemin loglanması sağlayarak geçmişe dönük kurtarmayı mümkün kılar.

4.2. Felaket Anı: Verinin Yanlışlıkla Silinmesi (Kaza)

Rapor ve video sunumunda "Kaza" olarak adlandırılan bu aşamada, veritabanı bütünlüğünü bozan yıkıcı bir işlem simüle edilmiştir:

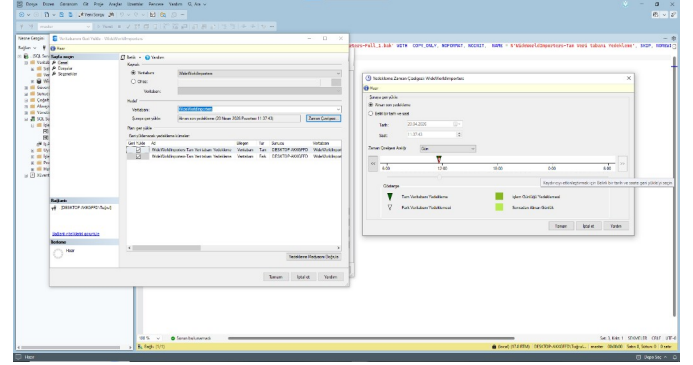
- **İşlem:** Query ekranında aşağıdaki komut çalıştırılarak `Sales.InvoiceLines` tablosu tamamen imha edilmiştir:

```
DROP TABLE Sales.InvoiceLines;
```

Bu işlemle birlikte tabloda yer alan tüm veriler ve tablo yapısı silinmiş, sistem "felaket" durumuna geçmiştir.

4.3. Kurtarma Süreci: Point-in-Time Restore (Zamanda Geriye Yolculuk)

Point-in-Time Restore, veritabanını sadece alınmış olan en son yedek anına değil, Transaction Log (İşlem Günlüğü) yedeklerini kullanarak felaketin gerçekleştiği saniyenin hemen öncesine döndürmeyi sağlayan ileri seviye bir kurtarma tekniğidir.



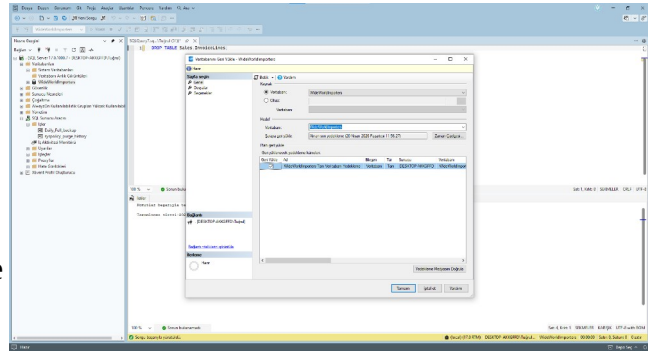
1. Veritabanı Geri Yükleme Arabirimi

Kurtarma işlemini başlatmak için SSMS (SQL Server Management Studio) üzerinde hedef veritabanına sağ tıklayarak **Tasks > Restore > Database** yolu izlenmiştir. Bu ekran, sistemin sahip olduğu tüm yedek setlerini otomatik olarak analiz ederek kurtarma planını oluşturur.

2. Teknik Yapılandırma: Timeline (Zaman Çizelgesi) Kullanımı

Felaketten kurtulmanın en can alıcı noktası, veritabanını hatanın (örneğin DROP TABLE komutunun) işlendiği andan kurtarmaktır.

- **Timeline Erişimi:** Restore penceresinin sağ üst köşesinde bulunan **Timeline** butonuna tıklanarak görsel zaman çizelgesi paneli açılmıştır.
- **Belirli Bir Ana Dönme:** "Specific date and time" seçeneği işaretlenerek, veritabanı felaketin gerçekleştiği andan (örneğin dün akşam saat 10:15 veya tablonun silindiği dakikadan 1 dakika öncesi) hemen önceki bir ana çekilmiştir.
- **Teknik Gereklilik:** Bu işlemin saniyeler bazında hassasiyetle çalışabilmesi için veritabanının **Recovery Model** ayarının mutlaka **Full** (Tam) olması gerekmektedir.



3. Erişim Engellerinin Aşılması (Exclusive Access)

Geri yükleme işlemi sırasında veritabanı dosyaları üzerinde tam kontrol (özel erişim) sağlanması gerekir. Eğer veritabanına bağlı bir kullanıcı veya açık bir sorgu ekranı varsa "Database in use" (Veritabanı kullanımda) hatası alınır. Bu teknik engeli aşmak için **Options** (Seçenekler) sekmesinde şu adımlar uygulanmıştır:

- **"Close existing connections to destination database"** (Hedef veritabanına olan mevcut bağlantıları kapat) kutucuğu işaretlenmiştir.
- **Mekanizma:** Bu seçenek işaretlendiğinde, SQL Server geri yükleme işlemine başlamadan önce veritabanına bağlı tüm aktif oturumları ve açık bağlantıları otomatik olarak sonlandırarak işlemi güvenli bir şekilde başlatır.

4. Sonuç ve Sistemin Doğrulanması

Yapılandırmalar tamamlandıktan sonra **OK** butonuna basılarak geri yükleme işlemi başlatılmıştır. İşlem sonucunda "Database restored successfully" mesajı alınarak veritabanının belirtilen zaman dilimine başarıyla döndürüldüğü teyit edilmiştir.

5. TEST YEDEKLEME SENARYOLARI: YEDEKLERİN DOĞRULUĞUNU TEST ETME

Bir Veritabanı Yöneticisi (DBA) için sadece yedekleme yapmak yeterli değildir; alınan yedeğin fiziksel olarak bozuk olmadığını ve ihtiyaç anında çalışabilir durumda olduğunu kanıtlamak, veri güvenliği stratejisinin en kritik adımıdır. Bu bölümde, "Yedeklerin doğruluğunu test etme" maddesi kapsamında hem T-SQL komutları ile teknik doğrulama hem de SSMS arayüzü üzerinden otomatik doğrulama yöntemleri uygulanmıştır.

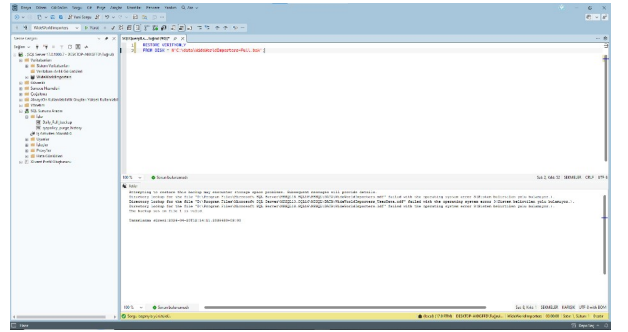
5.1. 1. Yöntem: T-SQL ile Teknik Doğrulama (RESTORE VERIFYONLY)

Bu yöntem, yedek dosyasını gerçekten geri yüklemekten (restore etmeden) dosya içindeki veri bloklarını tarayarak okunabilirliğini kontrol eder.

- **Sorgu Uygulaması:** SSMS üzerinde "New Query" penceresinde aşağıdaki komut çalıştırılmıştır:

```
RESTORE VERIFYONLY  
FROM DISK = N'C:\data\  
WideWorldImporters-Full.bak';
```

(Not: Dosya yolu, yedeğin fiziksel olarak bulunduğu dizine göre güncellenmelidir.)



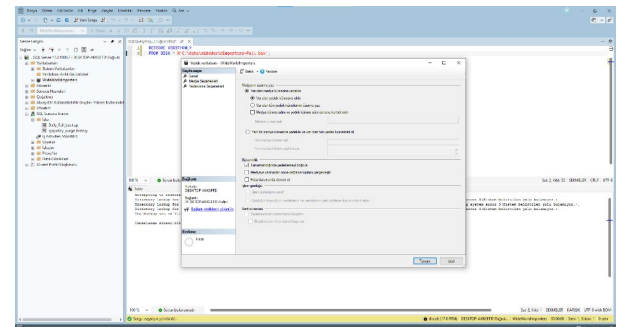
- **Sonuç ve Analiz:** Mesaj panosunda "**The backup set on device 1 is valid**" ifadesinin görülmesi, yedeğin fiziksel bütünlüğünün korunduğunu kanıtlar.

RESTORE VERIFYONLY komutu ile yedek dosyasının bütünlüğü (integrity) test edilmiş ve dosyanın geri yüklemeye hazır olduğu fiziksel olarak doğrulanmıştır.

5.2. 2. Yöntem: SSMS Arayüzü ile Otomatik Doğrulama

Yedekleme işlemi sırasında veya sonrasında görsel arayüz araçlarını kullanarak doğrulama yapmak için şu adımlar izlenmiştir:

1. **Erişim:** **Tasks > Back Up** penceresi açılmıştır.
2. **Yapılandırma:** Sol taraftaki **Options** (Seçenekler) sekmesine geçilmiştir.



3. **Güvenilirlik Ayarı: Reliability** (Güvenilirlik) bölümü altında yer alan "**Verify backup when finished**" (Bittiğinde yedeği doğrula) kutucuğu işaretlenmiştir.

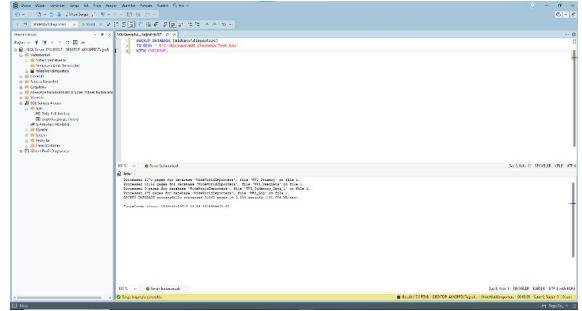
4. **İşlem:** Bu ayar sayesinde SQL Server, yedekleme işlemi biter bitmez otomatik olarak bütünlük testini gerçekleştirir.

5.3. 3. İleri Seviye Test: Sayfa Doğruluğu (Checksum)

Yedekleme işlemi sırasında verilerin diske yazılırken bozulma ihtimaline karşı matematiksel bir doğrulama katmanı eklemek amacıyla **Checksum** özelliği kullanılmıştır.

- **Kod Uygulaması:**

```
BACKUP DATABASE [WideWorldImporters]
TO DISK = N'C:\Backups\
WWI_Checksum_Test.bak'
WITH CHECKSUM;
```



Yedekleme sırasında kullanılan **WITH CHECKSUM** parametresi, verinin sayfalar bazında doğruluğunu kontrol ederek en üst düzeyde veri güvenliği sağlar.

6. SONUÇ

Proje 2: Veritabanı Yedekleme ve Felaketten Kurtarma Stratejileri kapsamında, kurumsal düzeydeki bir veritabanının (WideWorldImporters) iş sürekliliği standartlarına uygun olarak nasıl korunması gerektiği uygulamalı olarak simüle edilmiştir.

Bu proje, bir veritabanı yöneticisinin en öncelikli görevinin sadece veri depolamak değil, veriyi her türlü felaket senaryosuna (yanlışlıkla silme, yolsuzluk, sistem çökmesi) karşı erişilebilir kılmak olduğunu mühendislik disipliniyle ortaya koymuştur. Uygulanan stratejiler, sistemin "En Az Veri Kaybı" ve "En Hızlı Geri Dönüş" prensipleriyle çalışmasını mümkün kılmıştır.